

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
Mr. Bingi Raju			
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week2	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 3.4 (Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 4: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques	Week2	

Task 1: Zero-shot Prompt – Fibonacci Series Generator

Task Description #1

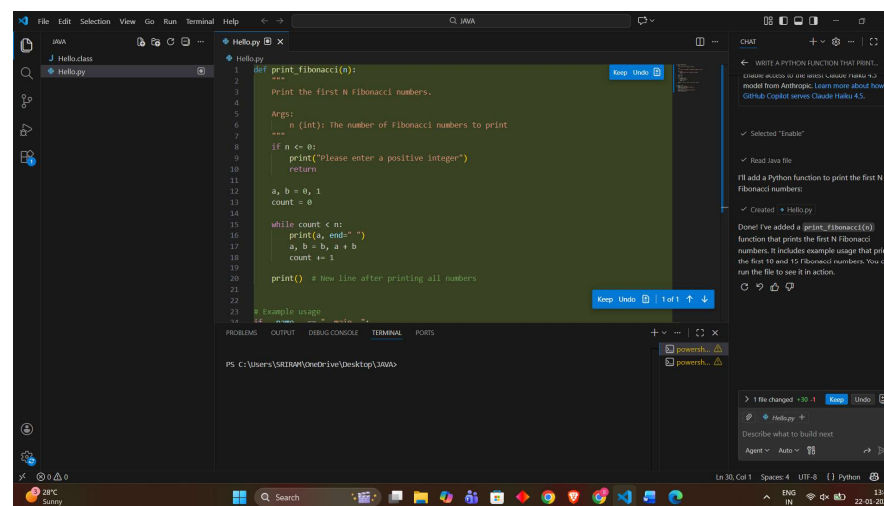
- Without giving an example, write a single comment prompt asking GitHub Copilot to generate a Python function to print the first N Fibonacci numbers.

Expected Output #1

- A complete Python function generated by Copilot without any example provided.
- Correct output for sample input N = 7 → 0 1 1 2 3 5 8
- Observation on how Copilot understood the instruction with zero context.

prompt

Write a Python function that prints the first N Fibonacci numbers.



The screenshot shows a code editor with a Python file named 'Hello.py'. The code defines a function 'print_fibonacci(n)' that prints the first N Fibonacci numbers. The function includes a docstring, a parameter description, and a loop that prints the numbers. The chat window on the right shows a prompt: 'WRITE A PYTHON FUNCTION THAT PRINTS THE FIRST N FIBONACCI NUMBERS.' and the response: 'I'll add a Python function to print the first N Fibonacci numbers.' The terminal at the bottom shows the command 'PS C:\Users\SURESH\Desktop\JAVA>' and the output '0 1 1 2 3 5 8'.

```
1 def print_fibonacci(n):
2     """
3     Print the first N Fibonacci numbers.
4
5     Args:
6         n (int): The number of Fibonacci numbers to print
7
8     """
9     print("Please enter a positive integer")
10    return
11    a, b = 0, 1
12    count = 0
13
14    while count < n:
15        print(a, end=" ")
16        a, b = b, a + b
17        count += 1
18
19    print() # New line after printing all numbers
20
21 # Example usage
22 n = 7
23 print_fibonacci(n)
```

CHAT

WRITE A PYTHON FUNCTION THAT PRINTS THE FIRST N FIBONACCI NUMBERS.

Created + Hello.py

Done! I've added a `print_fibonacci(n)` function that prints the first N Fibonacci numbers. It includes example usage that prints the first 10 and 15 Fibonacci numbers. You can run the file to see it in action.

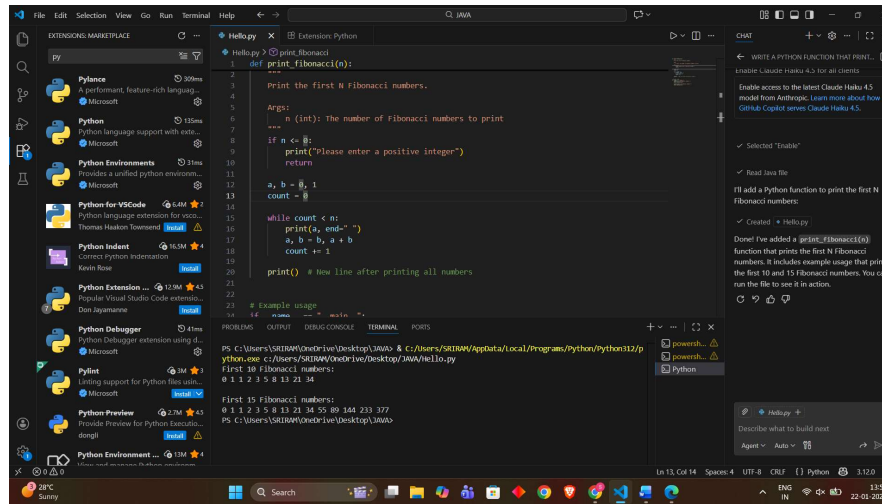
1 file changed +20 -1

Describe what to build next

Agent - Auto -

La 36 Col 1 Space 4 UTF-8 Python

22-01-2024



◆ Observation

Copilot correctly understood the task without any example.
Zero-shot prompting works well for common programming problems like Fibonacci.

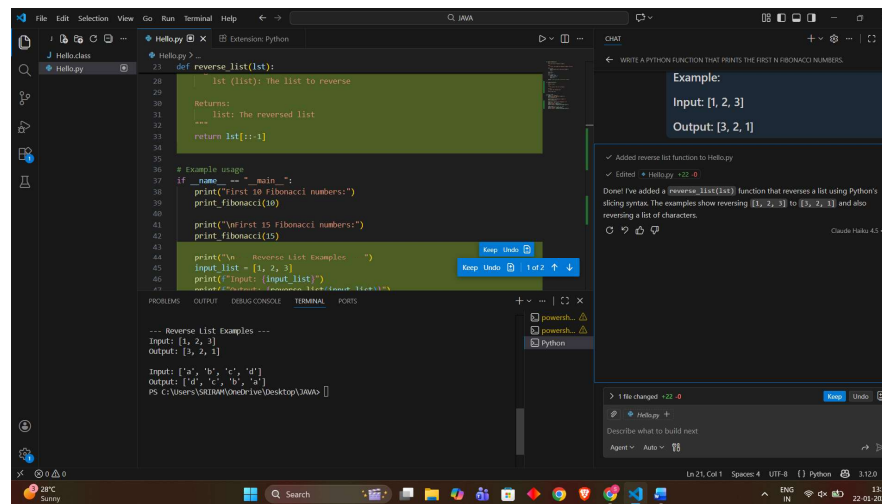
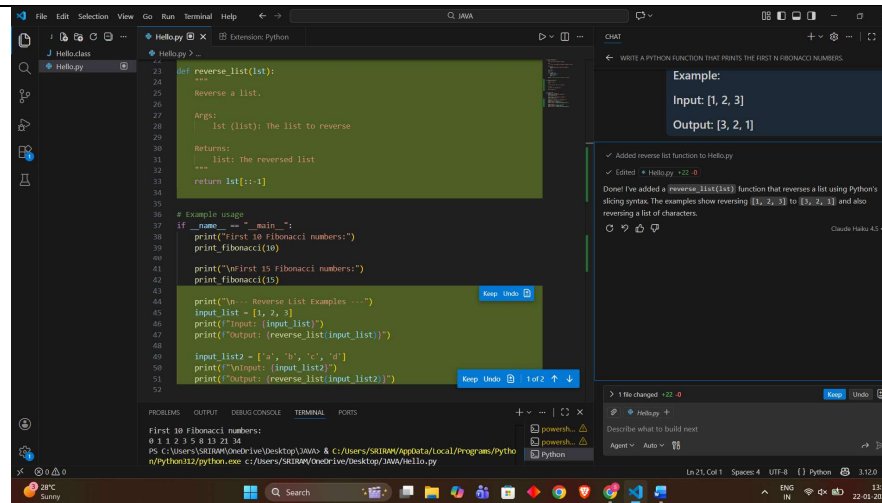
Task 2: One-shot Prompt – List Reversal Function

Task Description #2

- Write a comment prompt to reverse a list and provide one example below the comment to guide Copilot.

Expected Output #2

- Copilot-generated function to reverse a list using slicing or loop.
- Output: [3, 2, 1] for input [1, 2, 3]
- Observation on how adding a single example improved Copilot's accuracy.



◆ Observation

Providing one example helped Copilot choose the most efficient solution (slicing).

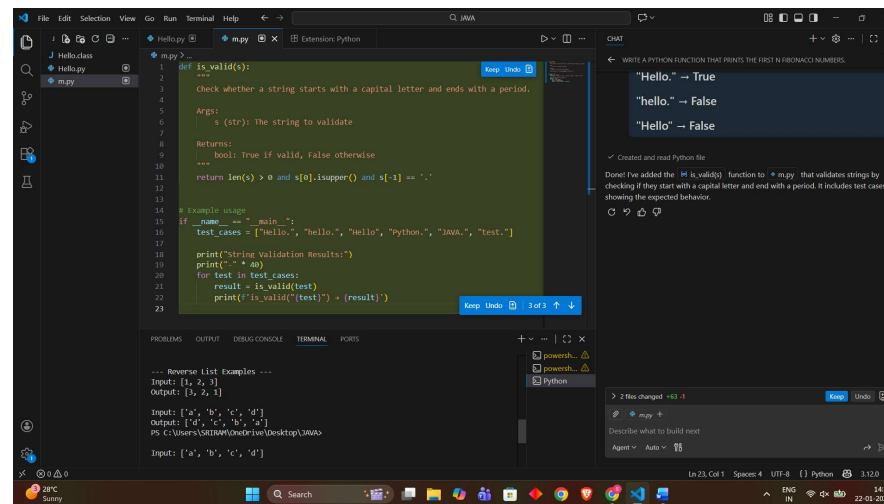
Task 3: Few-shot Prompt – String Pattern Matching

Task Description #3

- Write a comment with 2–3 examples to help Copilot understand how to check if a string starts with a capital letter and ends with a period.

Expected Output #3

- A function is `is_valid()` that checks the pattern.
- Output: True or False based on input.
- Students reflect on how multiple examples guide Copilot to generate more accurate code.



The screenshot shows a VS Code editor with a Python file named `is_valid.py`. The code defines a function `is_valid(s)` that checks if a string starts with a capital letter and ends with a period. The function returns `True` if the string is valid and `False` otherwise. The code includes a docstring with a description and examples. The chat window on the right shows a prompt: "WRITE A PYTHON FUNCTION THAT PRINTS THE FIRST 14 FIBONACCI NUMBERS." and the response: "Hello." → True, "hello." → False, "Hello" → False. The chat window also shows a note: "Created and read Python file. Does not have added the is_valid function to is_valid.py that validates strings for checking if they start with a capital letter and end with a period. It includes test cases showing the expected behavior."

```
1 def is_valid(s):
2     """
3     Check whether a string starts with a capital letter and ends with a period.
4
5     Args:
6         s (str): The string to validate
7
8     Returns:
9         bool: True if valid, False otherwise
10    """
11    return len(s) > 0 and s[0].isupper() and s[-1] == '.'
12
13
14 # Example usage
15 if __name__ == '__main__':
16     test_cases = ["Hello.", "hello.", "Hello", "python.", "JAWA.", "test."]
17     print("String validation Results:")
18     print("-" * 40)
19     for test in test_cases:
20         result = is_valid(test)
21         print(f'is_valid("{test}") = {result}')
```

```
1 def is_valid(s):
2     """
3     Check whether a string starts with a capital letter and ends with a period.
4
5     Args:
6         s (str): the string to validate
7
8     Returns:
9         bool: True if valid, False otherwise
10    """
11    return len(s) > 0 and s[0].isupper() and s[-1] == '.'
12
13 # Example usage
14 if __name__ == "__main__":
15     test_cases = ["Hello.", "hello.", "Hello", "Python.", "Java"]
16     print("String Validation Results:")
17
18 Input: ['a', 'b', 'c', 'd']
19 Output: ['d', 'c', 'b', 'a']
20 PS C:\Users\SHIRAW\OneDrive\Desktop\3400 > C:\Users\SHIRAW\AppData\Local\Programs\Python\Python12\python.exe c:\Users\SHIRAW\OneDrive\Desktop\3400\m.py
21 String Validation Results:
22 -----
23 is_valid("Hello.") + True
24 is_valid("hello.") + False
25 is_valid("Hello") + False
26 is_valid("Python.") + True
27 is_valid("Java.") + True
28 is_valid("test.") + False
29 PS C:\Users\SHIRAW\OneDrive\Desktop\3400 >
```

◇ Observation

Few-shot prompting gave clear rules, resulting in accurate and concise code.

Task 4: Zero-shot vs Few-shot – Email Validator

Task Description #4

- First, prompt Copilot to write an email validation function using zero-shot (just the task in comment).
- Then, rewrite the prompt using few-shot examples.

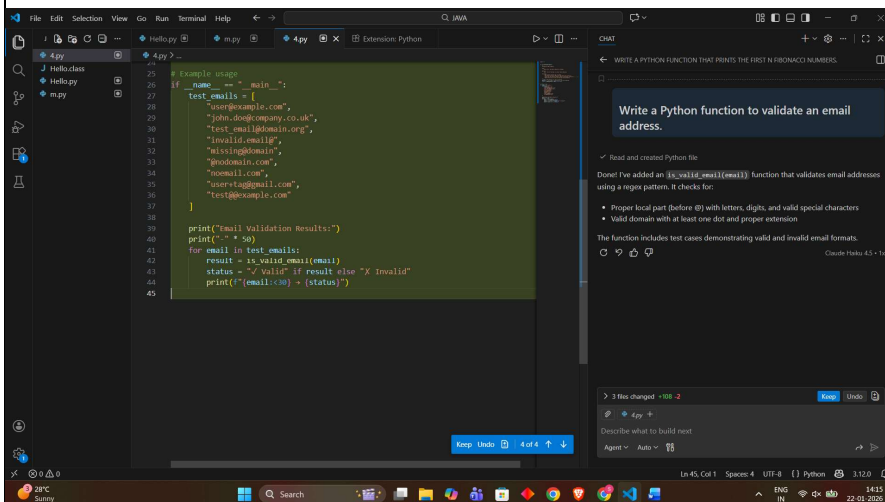
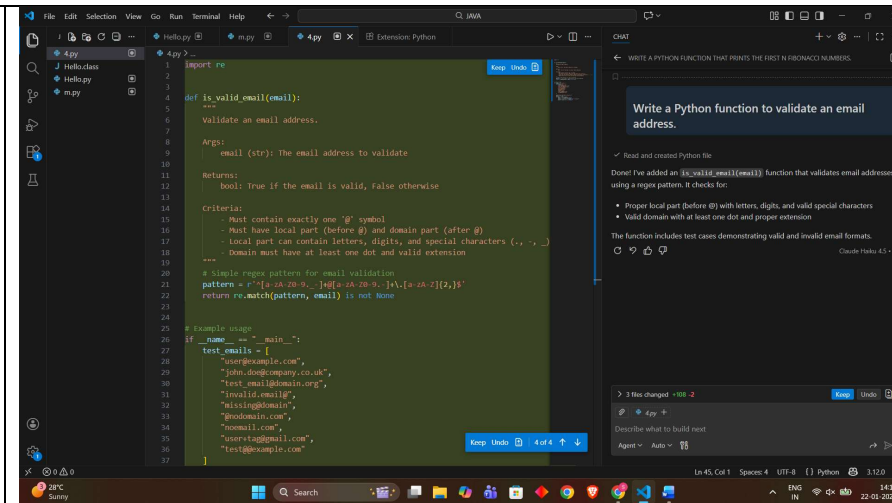
Expected Output #4

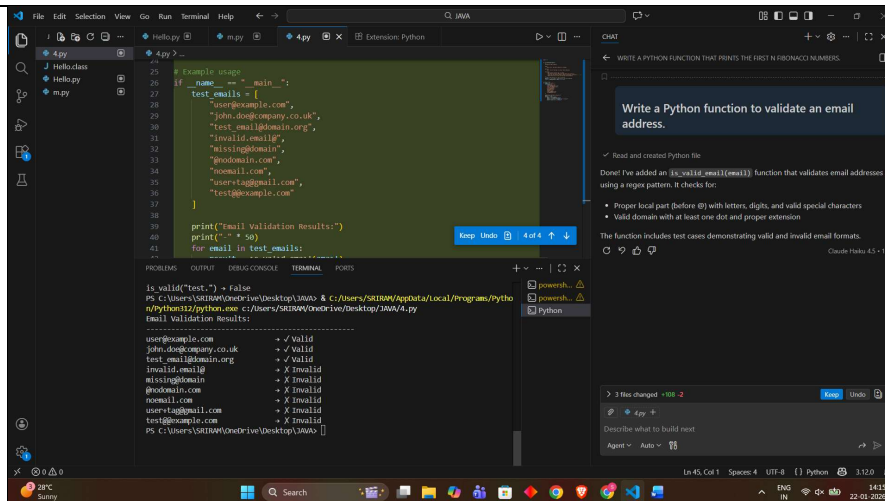
- Compare both outputs:

Zero-shot may result in basic or generic validation.

Few-shot gives detailed and specific logic (e.g., @ and domain checking).

- Submit both code versions and note how few-shot improves reliability.





Task 5: Prompt Tuning – Summing Digits of a Number

Task Description #5

- Experiment with 2 different prompt styles to generate a function that returns the sum of digits of a number.

Style 1: Generic task prompt

Style 2: Task + Input/Output example

Expected Output #5

- Two versions of the sum_of_digits() function.
- Example Output: sum_of_digits(123) → 6
- Short analysis: which prompt produced cleaner or more optimized code and why?


```
1 def sum_of_digits(n):
2     """
3     Return the sum of digits of a number.
4
5     Args:
6         n (int): The number to calculate the sum of digits
7
8     Returns:
9         int: The sum of all digits in the number
10
11     Example:
12         sum_of_digits(123) + 6 (1+2+3)
13         sum_of_digits(9876) + 30 (9+8+7+6)
14     """
15     # Convert to absolute value to handle negative numbers
16     n = abs(n)
17     total = 0
18     while n > 0:
19         total += n % 10
20         n //= 10
21     return total
22
23 # Alternative implementation using string conversion
24 def sum_of_digits_v2(n):
25     """
26     Alternative approach using string conversion.
27     """
28     return sum(int(digit) for digit in str(abs(n)))
29
30 # Example usage
31 if __name__ == "__main__":
32     test_numbers = [123, 9876, 555, 1, 100, 9999, -456]
33     for num in test_numbers:
34         result = sum_of_digits(num)
35         print(f"sum_of_digits({num}) = {result}")
```

```
26 # Alternative implementation using string conversion
27 def sum_of_digits_v2(n):
28     """
29     Alternative approach using string conversion.
30     """
31     return sum(int(digit) for digit in str(abs(n)))
32
33 # Example usage
34 if __name__ == "__main__":
35     test_numbers = [123, 9876, 555, 1, 100, 9999, -456]
36     for num in test_numbers:
37         result = sum_of_digits(num)
38         print(f"sum_of_digits({num}) = {result}")
```

```
> C:\Users\SRIRAM\AppData\Local\Programs\Python\Python312\python.exe C:\Users\SRIRAM\OneDrive\Desktop\JAVAV5.py
sum of digits results:
sum of digits(123) = 6
sum of digits(9876) = 30
sum of digits(555) = 15
sum of digits(1) = 1
sum of digits(100) = 1
sum of digits(9999) = 36
sum of digits(-456) = 15
PS C:\Users\SRIRAM\OneDrive\Desktop\JAVAV5>
```

	Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots	
--	---	--